

Document number: P3982R1
Date: 2026-03-20
Audience: Library Evolution Working Group
Reply-to: *Tomasz Kamiński* <tomaszkam at gmail dot com>
Mark Hoemmen <mark dot hoemmen at gmail dot com>

Split `strided_slice` into `extent_slice` and `range_slice` for C++26

1. Introduction

Addresses [PL007: Define the extent member of the strided slice](#)

This paper proposes two changes:

1. Rename `strided_slice` to `extent_slice` and adjust the meaning of its `extent` member, to designate the desired number of elements in the range produced by `submdspan`. **This change would be breaking after C++26 is shipped.**
2. Introduce a non-canonical `range_slice` slice type, that expresses the `(first, last, stride)` interface provided for range slicing in other programming languages. **While this is an extension that can be added in a later standard, we prefer to include it in C++26, as it gives `submdspan` a more ergonomic and familiar interface.**

Before	After
<code>std::strided_slice{0, 2, 3};</code>	<code>std::extent_slice{0, 1, 3};</code> <code>std::range_slice{0, 2, 3};</code>
<code>std::strided_slice{2, 10, 3};</code>	<code>std::extent_slice{2, 4, 3};</code> <code>std::range_slice{2, 12, 3};</code>

2. Revision history

2.2. Revision 1

- Changed title to better reflect the content.
- Remove the proposal to support any type decomposable into `[first, last, stride]`, as slice type. This change will be proposed for C++29.
- Corrected and updated the wording for `strided_slice` per Charles Husson's suggestions.

2.1. Revision 0

Initial revision.

3. Motivation and Scope

The invocation `smd = submdspan(md, strided_slice{offset, extent, stride})` refers to two ranges of indices:

- *input span*: size of the range of indices into `md`, that can be accessed by `smd`; and
- *output extent*: size of the range of indices that are valid indices for `smd`.

Given the above, there are two possible interpretations of the `extent` for the above example:

- *input span*, thus *output extent* is $1 + (\text{extent} - 1) / \text{stride}$; or
- *output extent*, thus *input span* is $\text{extent} * \text{stride}$.

In most cases, this two meanings are functionally equivalent and they can be transformed into each other. However, due use of the division in the *input span* interpretation does not support the following:

- `stride` whose value is zero, that could be used to produce non-unique layouts; or
- specifying the value of *output extent* statically, while keeping the `stride` dynamic.

As `strided_slice` is used as one of the canonical forms of the slices, we propose that the `strided_slice::extent` member should represent the *output extent*.

3.1. Introducing `range_slice`

One argument for using the *input span* as the value of `stride_slice::extent` was consistency with other programming languages' range slicing interfaces. However, surveying the slicing interface in common languages shows that they all use `first`, `last` instead of `offset`, `length`.

- [Fortran](#): `array(first:last)` and `array(first:last:step)`
- [python](#): `array[first:last]` and `array[first:last:step]`
- [Matlab](#): `array(first:last)` and `array(first:step:last)`
- [rust](#): `array[first..last]`, `array[first..]`, `array[..last]` and `array[..]`.

Based on the intuition built from other languages, `submdspan(md, strided_slice{2, 5, 1})`, should select elements `[2, 5)`, instead of `[2, 7)` as currently specified.

To provide a interface consistent with existing practice in many languages, we propose to introduce a new vocabulary type for "range" slice:

```
template<typename FirstType, typename LastType, typename StrideType = constant_wrapper<1zu>>
struct range_slice
{
    [[no_unique_address]] FirstType first{};
    [[no_unique_address]] LastType last{};
    [[no_unique_address]] StrideType stride{};
};
```

3.2. Expressing required values directly

Creating a subset of a multidimensional index space (`submdspan`) requires the *output extent* to be known. In the model where user provides an *input span*, the *output extent* computation must be performed, and is thus duplicated in each layout.

In contrast, with this paper's proposed changes, the members of `strided_slice` directly represent values used by `submdspan` creation. This gives programmers more direct control over the process. In particular, in cases when the value of *output extent* is known, or can be reused between invocations, passing it directly can lead to measurable speed-up. We illustrate this by including benchmark results below.

submdspan(md, X{0, span, stride})		
	Before (X is strided_slice)	After (X is extent_slice)
stride	span = 10	span = 1 + 9 / stride
3	3.03 ns	1.52 ns
1	3.03 ns	1.51 ns
<code>std::cw<3></code>	1.01 ns	1.01 ns
<code>std::cw<1></code>	1.02 ns	1.01 ns

As previously mentioned, `range_slice` can be used if passing an *input span* is preferred. Results from a benchmark similar to one used above show no significant performance difference.

submdspan(md, range_slice{0, 10, stride})		
stride	Before	After
3	3.03 ns	3.03 ns
1	3.03 ns	1.52 ns
<code>std::cw<3></code>	1.01 ns	1.01 ns
<code>std::cw<1></code>	1.01 ns	1.01 ns

More details about above results may be found [here](#).

3.3. Example of static *output extent* usage

As mentioned before, the current *input span* specification does not give users a way to select a statically sized subset of elements with dynamic stride. For example, imagine that we want to select 5 elements, evenly spaced from the `mdspan md`.

With the proposed change, we can simply express that as:

```
auto smd = sumdspan(md, strided_slice{cw<0>, cw<5>, md.extents()[0] / 5})
```

Note that the number of elements in the `smd` is always known statically, regardless if the source span had static extents.

3.4. Example of zero stride value usage

Using a zero as the value of the `stride` leads to a non-unique layout mapping, because incrementing the index does not change the referenced element. Thus, Standard layout mappings like `layout_stride::mapping`, `layout_left_padded::mapping`, and `layout_right_padded::mapping` do not accept zero strides as input to their constructors.

However, `submdspan` can be also used with `mdspan` with user-defined mappings that are not required to be unique. Any mapping can be queried (via `is_always_unique` or `is_unique`) for this property.

One could imagine a `layout_stride_relaxed`¹ layout that is equivalent to `layout_strided`, except that it does not require that the provided strides result in a non-unique mapping. In case of `mdspan` with such mapping, a zero stride may be used to create a layout that "broadcasts" a single element over the given extent.

```
auto smd = submdspan(md, strided_slice{3, 5, 0});
```

For the above example, each of `smd[0]`, `smd[1]`, ..., `smd[4]` results in a reference to `md[3]`.

As mentioned before, such a slice specification is not representable currently. While the paper does not lift the current requirements on the stride value being non-zero, it permits zero strides in the future for a subset of mappings.

¹ A version of `layout_stride_relaxed` was [recently merged into](#) NVIDIA's CCCL library.

3.5. Rename `strided_slice` to `extent_slice`

After expanding the set of accepted slice types, the `strided_slice` name does not capture the difference well, as `range_slice` is also strided. Thus, we propose to rename the class to `extent_slice`.

The `extent_slice` name was selected to focus on the fact that its members define the size (extent) of the produced (output) multidimensional index space. That is, it directly reflects the value of `smd.extent(k)`, where `smd` is the `mdspan` produced by `submdspan`, and `k` is the index of the extent to which the slice is applied.

We also avoid using words commonly used to refer to the range like "size," "length" (as in `offset + length`), or "span." The word "size" is particularly overloaded, for example in the `std::ranges::sized_range` concept.

3.6. Example showing all the changes

Here is an example showing all the proposed changes.

- Status quo
 - `s = strided_slice{1, 10, 3}` views input indices 1, 4, 7, and 10.
 - The input span is `[s.offset, s.offset + s.extent) = [1, 1+10) = [1, 11)`.
 - The output extent is `s.offset + (s.extent - 1) / s.stride = 1 + (10 - 1) / 3 = 4`, which is NOT the same as `s.extent`.
- Proposed change
 - `s = extent_slice{1, 10, 3}` views input indices 1, 4, 7, 10, 13, 16, 19, 22, 25, and 28.
 - The input span is `[s.offset, s.offset + 1 + (s.extent - 1) * s.stride) = [1, 29)`.
 - The output extent is 10, which is the same as `s.extent`.
 - `range_slice{1, 11, 3}` views input indices 1, 4, 7, and 10.

4. Ship vehicle, proposed polls, and review history

This paper proposes two changes:

1. Rename `strided_slice` to `extent_stride` and adjust the meaning of `extent` member, to designate the desired number of elements in the produced range.
2. Introduce a new non-canonical slice type `range_slice`, that expresses the `(first, last, stride)` interface for range slicing provided by other programming languages.

From the above only the **first change** need to be applied in the C++26 timeframe. However, we prefer that the second change be included in C++26 as well, as it makes `submdspan` more consistent with slicing facilities in other programming languages.

4.1. Proposed polls

1. Forward P3982R1 as resolution for PL007 NB comment to LWG for inclusion to C++26.
2. Forward P3982R1 without `range_slice` as resolution for PL007 NB comment to LWG for inclusion to C++26.

4.2. Nonbinding LEWG review

LEWG reviewed R0 of this proposal on 2026-03-03 and took the following polls. The review and poll results were nonbinding for procedural reasons. LEWG must review this proposal again at the Croydon meeting later in March 2026.

POLL (nonbinding): We want to pursue adding `range_slice` to C++26.

SF	F	N	A	SA
10	6	0	0	0

- Attendance: 22
- Authors' opinion: 2xSF
- Outcome: Strong consensus in favor

POLL (nonbinding): We want to pursue adding 3 element decomposition for C++26.

SF	F	N	A	SA
3	3	4	2	0

- Attendance: 22
- Authors' opinion: 2xSF
- Outcome: very weak consensus

The authors took these poll results as guidance to delay adding the 3-element decomposition feature to C++29. R1 includes this revision.

4.3. `strided_slice` needs to target C++26

`strided_slice` is one of the canonical slice types that define the interface between `submdspan` (and potentially other components providing such facility) and custom layouts. Thus, it is important that the interface is both minimal (reducing the burden on layout implementers) and able to represent a wide range of input without loss of information. As this document explains, the current specification of `strided_slice` fails in both accounts. It incurs the cost of division, and it cannot be used for non-unique layouts.

We currently reserve the right to introduce additional canonical stride types. We could imagine introducing a separate `canonical_strided_slice` type in a later standard. However, in contrast to amending `strided_slice`, this would essentially duplicate the number of types that layouts would need to handle, as pre-existing code depending on sliceable layout requirements may still produce `strided_slice` objects.

4.4. `range_slice` should target C++26

While `range_slice` could be added later as an extension, the authors strongly believe that the ergonomics of `submdspan` would be severely degraded, without the ability to specify a slice using an *input span*.

During the work on this paper, one of the authors (Tomasz) made the following mistakes, when transforming the *input span* to an *output extent*:

- using `span / stride`,
- using `(span - 1) / stride` (missing +1),
- not accounting for span equal to zero.

This shows that even for programmers familiar with the topic, such computations remain bug-prone.

5. Impact and Implementability

This paper only impacts the behavior of the `std::submdspan` library function that was introduced in C++26.

Here is a [patch series](#) implementing the proposed wording changes to `submdspan` in `libstdc++`.

6. Proposed Wording

The proposed wording changes refer to N5032 (C++ Working Draft, 2025-12-15).

Apply following changes to section [mdspan.syn] Header <mdspan> synopsis:

```
// [mdspan.sub], submdspan creation
template<class OffsetType, class LengthType, class StrideType>
struct strided_slice;
template<class FirstType, class LastType,
        class StrideType = std::constant_wrapper<1zu>>
struct range_slice;

template<class LayoutMapping>
struct submdspan_mapping_result;
```

Apply following changes to section [mdspan.sub.overview] Overview :

-1- The submdspan facilities create a new mdspan viewing a subset of elements of an existing input mdspan. The subset viewed by the created mdspan is determined by the SliceSpecifier arguments.

-2- Given a signed or unsigned integer type IndexType, a type S is a *submdspan slice type for IndexType* if at least one of the following holds:

- -2.1- is_convertible_v<S, full_extent_t> is true;
- -2.2- is_convertible_v<S, IndexType> is true;
- -2.3- S is a specialization of strided_slice and is_convertible_v<X, IndexType> is true for X denoting S::offset_type, S::extent_type, and S::stride_type; ~~or~~
- -2.2- S is a specialization of range_slice and is_convertible_v<X, IndexType> is true for X denoting the type of the S::first, S::last, and S::stride members; or
- -2.4- all of the following hold:
 - -2.4.1- the declaration auto [...ls] = std::move(s); is well-formed for some object s of type S,
 - -2.4.2- sizeof...(ls) is equal to 2, and
 - -2.4.3- (is_convertible_v<decltype(std::move(ls)), IndexType> && ...) is true.

-3- Given a signed or unsigned integer type IndexType, a type S is a *canonical submdspan index type for IndexType* if S is either IndexType or constant_wrapper<v> for some value v of type IndexType, such that v is greater than or equal to zero.

-4- Given a signed or unsigned integer type IndexType, a type S is a *canonical submdspan slice type for IndexType* if exactly one of the following is true:

- -4.1- S is full_extent_t;
- -4.2- S is a canonical submdspan index type for IndexType; or
- -4.3- S is a specialization of strided_slice where all of the following hold:
 - -4.3.1- S::offset_type, S::extent_type, and S::stride_type are all canonical submdspan index types for IndexType; and
 - -4.2.2- if S::stride_type and S::extent_type are both specializations of constant_wrapper, then S::stride_type::value is greater than zero.

-5- A type S is a *collapsing slice type* if [...]

-6- A type S is a *unit-stride slice type* if [...]

-7- Given an object e of type E that is a specialization of extents, and an object s of type S that is a canonical submdspan slice type for E::index_type, the *submdspan slice range of s for the k^{th} extent of e* is:

- -7.1- [0, e.extent(k)), if S is full_extent_t;
- -7.2- [E::index_type(s.offset), E::index_type(s.offset)), if S is a specialization of strided_slice and E::index_type(s.extent) is zero; otherwise
- -7.2- [E::index_type(s.offset), E::index_type(s.offset + 1 + (s.extent - 1) * s.stride)), if S is a specialization of strided_slice; otherwise
- -7.3- [E::index_type(s), E::index_type(s)+1)

-8- Given a type E that is a specialization of extents, a type S is a *valid submdspan slice type for the k^{th} extent of E* if S is a canonical slice type for E::index_type, and for x equal to E::static_extent(k), either x is equal to dynamic_extent; or

- -8.1- if S is a specialization of strided_slice:

- -8.1.1- ~~if `S::offset_type` is a specialization of `constant_wrapper`, then `S::offset_type::value` o is less than or equal to x ;~~
- -8.1.2- ~~if `S::extent_type` is a specialization of `constant_wrapper` then `S::extent_type::value` e is less than or equal to x ;~~
- -8.1.2- if e is greater than one, then t is greater than zero.
- -8.1.3- ~~if both `S::offset_type` and `S::extent_type` are specializations of `constant_wrapper` then, `S::offset_type::value` + `S::extent_type::value` if e is greater than zero, then $o + 1 + (e - 1)t$ is less than or equal to x~~

where

- -8.1.4- o is the value of `S::offset_type::value` if `S::offset_type` is a specialization of `constant_wrapper` and 0 otherwise,
- -8.1.4- e is the value of `S::extent_type::value` if `S::extent_type` is a specialization of `constant_wrapper` and 0 otherwise,
- -8.1.4- t is the value of `S::stride_type::value` if `S::stride_type` is a specialization of `constant_wrapper` and 1 otherwise.
- -8.2- if `S` is a specialization of `constant_wrapper`, then `S::value` is less than x

-9- Given an object e of type `E` that is a specialization of `extents` and an object s of type `S`, s is a valid *submdspan slice* for the k^{th} extent of e if:

- -9.1- `S` is a valid *submdspan slice type* for k^{th} extent of `E`;
- -9.2- the k^{th} interval of e contains the *submdspan slice range* of s for the k^{th} extent of e ; and
- -9.3- if `S` a specialization of `strided_slice` then:
 - -9.3.1- $s.extent$ is greater than or equal to zero, and
 - -9.3.2- either $s.extent$ ~~equals zero~~ is less than two or $s.stride$ is greater than zero.

Apply following changes to section [mdspan.sub.strided.slice] `strided_slice`:

23.7.3.7.2 ~~strided_slice~~ Range slices

[mdspan.sub.strided.slice]

-1- `strided_slice` and `range_slice` represents a set of extent regularly spaced integer indices. The indices start at offset and first respectively, and increase by increments of `stride`.

```
namespace std {
    template<class OffsetType, class ExtentType, class StrideType>
    struct strided_slice {
        using offset_type = OffsetType;
        using extent_type = ExtentType;
        using stride_type = StrideType;

        [[no_unique_address]] offset_type offset{};
        [[no_unique_address]] extent_type extent{};
        [[no_unique_address]] stride_type stride{};
    };
};
```

```
template<class FirstType, class LastType,
class StrideType = std::constant_wrapper<1zu>>
struct range_slice {
    [[no unique address]] FirstType first{};
    [[no unique address]] LastType last{};
    [[no unique address]] StrideType stride{};
};
```

-2- `strided_slice` and `range_slice` ~~has~~ have the data members and special members specified above. ~~It has~~ They have no base classes or members other than those specified.

-3- *Mandates:* `OffsetType`, `ExtentType`, `FirstType`, `LastType`, and `StrideType` are signed or unsigned integer types, or model *integral-constant-like*.

[*Note:* Both `strided_slice{ .offset = 1, .extent = 104, .stride = 3}` and `range_slice{ .first = 1, .last = 11, .stride = 3}` indicates the indices 1, 4, 7, and 10. Indices are selected from the half-open interval $[1, 1 + 10)$. — *end note*]

Apply following changes to section [mdspan.sub.helpers] Exposition-only helpers:

```
template<class T>
concept is-strided-slice = see below;
```

-2- The concept *is-strided-slice*<T> is satisfied and modeled if and only if T is a specialization of *strided_slice*.

```
template<class T>
concept is-range-slice = see below;
```

-?- The concept *is-range-slice*<T> is satisfied and modeled if and only if T is a specialization of *range_slice*.

```
template<class IndexType, class S>
constexpr auto canonical-index(S s);
```

-3- *Mandates*: [...];

-4- *Preconditions*: [...];

-5- *Effects*: [...];

```
template<class IndexType, class OffsetType, class SpanType, class... StrideTypes>
constexpr auto canonical-range-slice(OffsetType offset, SpanType span, StrideTypes... strides);
```

-?- Let:

- StrideType be constant_wrapper<IndexType(1)> if sizeof...(StrideTypes) == 0 or SpanType denotes constant_wrapper<IndexType(0)>, and StridesTypes...[0] otherwise;
- stride be
 - StrideType() if StrideType is a specialization of constant_wrapper, otherwise
 - IndexType(1) if span == 0 is true, otherwise
 - strides...[0];
- extent-value be 1 + (span - 1) / stride if span != 0 is true, and 0 otherwise;
- extent be cw<IndexType(extent-value)> if both SpanType and StrideType are specializations of constant_wrapper, and IndexType(extent-value) otherwise.

-?- Mandates:

- sizeof...(StrideTypes) <= 1 is true, and
- if StrideType is specialization of constant_wrapper, then StrideType::value > 0 is true.

-?- Preconditions: stride > 0 is true.

-?- Returns: strided_slice{ .offset = offset, .extent = extent, .stride = stride };

```
template<class IndexType, class S>
constexpr auto canonical-slice(S s);
```

-6- *Mandates*: S is a submdspan slice type for IndexType.

-7- *Effects*: Equivalent to:

```
if constexpr (is_convertible_v<S, full_extent_t>) {
    return static_cast<full_extent_t>(std::move(s));
} else if constexpr (is_convertible_v<S, IndexType>) {
    return canonical-index<IndexType>(std::move(s));
} else if constexpr (is-strided-slice<S>) {
    return strided_slice{
        .offset = canonical-index<IndexType>(std::move(s.extent)),
        .extent = canonical-index<IndexType>(std::move(s.offset)),
        .stride = canonical-index<IndexType>(std::move(s.stride))
    };
}
auto c_extent = canonical-index<IndexType>(std::move(s.extent));
auto c_offset = canonical-index<IndexType>(std::move(s.offset));
if constexpr (is_same_v<decltype(c_extent), constant_wrapper<IndexType(0)>>) {
    return strided_slice{
        .offset = c_offset,
        .extent = c_extent,
        .stride = cw<IndexType(1)>
    };
} else {
    return strided_slice{
        .offset = c_offset,
```

```

        .extent = c_extent,
        .stride = canonical_index<IndexType>(std::move(s.stride))
    },
}
    } else if constexpr (is_range_slice<S>) {
        auto c_first = canonical_index<IndexType>(std::move(s.first));
        auto c_last = canonical_index<IndexType>(std::move(s.last));
        return canonical_slice_range<IndexType>(
            c_first,
            canonical_index<IndexType>(c_last - c_first),
            canonical_index<IndexType>(std::move(s.stride)));
    } else {
        auto [s_first, s_last] = std::move(s);
        auto c_first = canonical_index<IndexType>(std::move(s_first));
        auto c_last = canonical_index<IndexType>(std::move(s_last));
        return strided_slice{
            .offset = c_first,
            .extent = canonical_index<IndexType>(c_last - c_first),
            .stride = cw<IndexType>(1)
        },
        return canonical_slice_range<IndexType>(
            c_first,
            canonical_index<IndexType>(c_last - c_first));
    }
}

```

Apply following changes to section [mdspan.sub.extents] submdspan_extents function:

```

template<class IndexType, size_t... Extents, class... SliceSpecifiers>
constexpr auto submdspan_extents(const extents<IndexType, Extents...>& src,
                                SliceSpecifiers... raw_slices);

```

-1- Let slices be [...];

-2- Constraints: [...];

-3- Mandates: [...];

-4- Preconditions: [...];

-5- Let SubExtents be a specialization of extents such that:

- -5.1- SubExtents::rank() equals $MAP_RANK(slices, Extents::rank())$; and;
- -5.2- for each rank index k of ~~Extents~~ extents<IndexType, Extents...> such that the type of slices... $[k]$ is not a collapsing slice type, SubExtents::static_extent($MAP_RANK(slices, k)$) equals the following, where Σ_k denotes the type of slices... $[k]$:
 - -5.2.1- Extents::static_extent(k) if Σ_k denotes the full_extent_t; otherwise
 - ~~-5.2.2- 0, if Σ_k is a specialization of strided_slice and $\Sigma_k::extent_type$ denotes constant_wrapper<IndexType>(0); otherwise~~
 - -5.2.3- $1 + ((\Sigma_k::extent_type::value - 1) / \Sigma_k::stride_type::value)$ $\Sigma_k::extent_type::value$ if Σ_k is a specialization of strided_slice whose extent_type ~~and stride_type~~ denotes specializations of constant_wrapper;
 - -5.2.4- otherwise, dynamic_extent.

-6- Returns: A value ext of type SubExtents such that for each index k of extents<IndexType, Extents...>, where the type of slices... $[k]$ is not a collapsing slice type, ext.extent($MAP_RANK(slices, k)$) equals the following where σ_k denotes slices... $[k]$:

- -6.1- ~~$\sigma_k.extent == 0 ? 0 : 1 + (\sigma_k.extent - 1) / \sigma_k.stride$~~ if the type of σ_k is specialization of strided_slice,
- -6.2- otherwise, $U-L$, where $[L, U)$ is the submdspan slice range of σ_k for the k^{th} extent of src.

Apply following changes to section [mdspan.sub.map.common] Common:

-6- Let `sub_strides` be an array<SubExtents::index_type, SubExtents::rank()> such that for each rank index k of `extents()` for which the type of `slices...[k]` is not a collapsing slice type, `sub_strides[MAP_RANK(slices,k)]` equals:

- -6.1- `stride(k) * s.stride` if type of `s` is a specialization of `strided_slice` and ~~`s.stride < s.extent`~~ `s.extent > 1` is true, where `s` is `slices...[k]`;
- -6.2- otherwise, `stride(k)`.

Replace all occurrences of `strided_slice` and *`is-strided-slice`* in `[mdspan.sub]` with `extent_slice` and *`is-extent-slice`* respectively.

Update the value of the `__cpp_lib_submdspan` in `[version.syn]` Header <version> synopsis to reflect the date of approval of this proposal.

7. Acknowledgements

Christian Trott and Charles Hussong offered many useful suggestions and corrections to the proposal.

8. References

1. Poland, "PL007 23.7.3.7 `[mdspan.sub]` `[mdspan.sub]` Define the extent member of the `strided_slice`", (PL007, <https://github.com/cplusplus/nbballot/issues/816>)
2. Tomasz Kamiński, "[RFC v2 0/4] libstdc++: Implement P3982R1 `range_slice` and", (<https://gcc.gnu.org/pipermail/libstdc++/2026-March/065843.html>)
3. Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, and Nevin Liber, "Submdspan", (P2630R4, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2630r4.html>)
4. Thomas Köppe, "Working Draft, Standard for Programming Language C++" (N5032, <https://wg21.link/n5032>)